TECHNICAL SPECIFICATION / IMPLEMENTATION GUIDE / V1.0

Sky
CLI Agent

A Node.js / TypeScript Command-Line AI Agent with Interactive TUI, Multi-Provider LLM Support, and Production-Grade Safety Workflows

This document defines the complete technical specification for Sky, an open-source CLI agent that mirrors the Cursor developer experience while operating as a standalone, brand-independent product. It covers system architecture, module breakdown, command interface, tool implementations, session and configuration management, LLM provider integration, safety workflows, testing strategy, error handling, documentation requirements, and a phased development roadmap. The specification is intended for senior engineers and is detailed enough to begin implementation immediately without further clarification.

DOCUMENT	Sky CLI Agent Technical Specification
VERSION	1.0 · Final
DATE	2026-07-12
AUDIENCE	Senior Engineering Team · Architects · Contributors
STATUS	Production-Ready Specification

Table of Contents

1. Project Overview

Sky is an open-source command-line interface (CLI) agent designed to bring the interactive AI-assisted development experience pioneered by Cursor to the terminal. It is built as a standalone, brand-independent product: it does not depend on Cursor binaries, services, or proprietary APIs, and it is distributed under a permissive open-source license. The agent runs entirely on the user's machine, communicates directly with one or more large language model (LLM) providers, and operates on the local filesystem and shell with explicit, auditable user consent.

The project's reason for existing is straightforward: developers who live in the terminal deserve the same quality of AI-assisted coding, file manipulation, and shell orchestration that graphical editors have offered since 2023, without being forced to leave their workflow, adopt a new editor, or surrender their data to a cloud-hosted agent. Sky delivers this by combining a React-based terminal user interface (TUI) built on Ink, a modular tool system, a provider-agnostic LLM abstraction, and a conservative safety model that requires human approval for any destructive or irreversible action.

1.1 Positioning and Scope

Sky is positioned as a general-purpose developer agent, not a vertical tool. It is equally useful for refactoring a TypeScript monorepo, scaffolding a new microservice, investigating a flaky test, drafting commit messages from a diff, or answering ad-hoc questions about a codebase. The scope explicitly includes:

- **Interactive agentic coding** — the agent reads, writes, edits, and searches files, executes shell commands, and interacts with git, all under user supervision.
- **Plan-first workflows** — a dedicated *Plan* mode in which the agent clarifies intent, proposes a design, and only executes after explicit approval.
- **Read-only inquiry** — an *Ask* mode that answers questions without mutating the workspace, suitable for code exploration and learning.
- **Headless automation** — a non-interactive CI mode (the --yolo and --force flags) for running the agent inside pipelines, with all approvals pre-granted and every action logged to an audit trail.
- **Session persistence** — every interactive session is checkpointed to disk and can be resumed, branched, or inspected after the fact.

The scope explicitly excludes: hosting a web UI, providing a graphical diff viewer, embedding an editor component, integrating with cloud storage providers, and offering a hosted/SaaS variant. Sky is a local-first CLI tool; if a feature cannot be delivered without a remote server under the user's control, it is out of scope for v1.

1.2 Design Principles

Five principles govern every architectural decision in Sky. They are listed in priority order; when two principles conflict, the earlier one wins.

1. **Local-first, user-sovereign.** The agent runs on the user's hardware, reads files the user can see, and sends only the data the user authorizes to LLM providers. No telemetry is collected without opt-in, and no code leaves the machine except as prompts to the configured provider.
2. **Consent by default.** Any action that mutates the filesystem, executes a shell command, or pushes to a remote must be explicitly approved. The agent never assumes consent; the user can grant it per-action, per-session, or globally via configuration.
3. **Modular to the bone.** Every concern — LLM communication, tool execution, session storage, UI rendering, configuration — lives in its own module with a typed interface. Modules can be replaced, mocked, or extended without touching the core.
4. **Observable in production.** Every tool call, every LLM request, every approval decision, and every error is logged with structured context. A user or operator can always reconstruct what the agent did and why.
5. **Faithful to the terminal.** The UI uses the terminal's strengths: monospace alignment, color, keyboard-driven interaction, and composability with unix pipes. Sky does not try to be an IDE; it tries to be the best possible terminal citizen.

1.3 Target Users

Persona	Workflow	Primary Mode	Key Need
Backend engineer	Refactoring services, writing tests, reviewing diffs	Agent	Fast, safe, multi-file edits with approval gates
DevOps / SRE	Investigating incidents, running diagnostics, drafting runbooks	Agent + Shell	Controlled shell execution with audit trail
Open-source maintainer	Triage issues, reproduce bugs, draft changelogs	Plan + Ask	Plan-first exploration before any change
CI/CD pipeline	Automated dependency upgrades, lint fixes, doc generation	Headless (--yolo)	Deterministic, logged, exit-code-driven runs
New team member	Exploring an unfamiliar codebase, asking questions	Ask	Read-only, no risk of breaking anything
Security reviewer	Auditing what an agent did during a session	sky ls + sky resume --view	Replayable, inspectable session history

1.4 Success Criteria

A v1.0 release of Sky is considered successful when all of the following are true. These criteria are tracked as launch gates and reviewed at every release candidate.

Criterion	Measurement	Target
Core UX parity	User can complete a 5-file refactor task end-to-end	100% completion in interactive TUI
Provider coverage	Number of LLM providers with first-class adapters	≥ 4 (OpenAI, Anthropic, Ollama, OpenRouter)
Safety record	Destructive actions taken without explicit approval	0 across all test sessions
Session integrity	Resume rate after process crash or Ctrl-C	100% recovery with no data loss
Test coverage	Line coverage across agent/, tools/, session/, config/	≥ 85%
Cold-start latency	Time from sky invocation to first prompt	< 800 ms on M1 class hardware
Headless determinism	Same prompt + same repo → same file changes	Bit-identical diff in 95%+ of runs
Documentation completeness	All public commands, flags, and config keys documented	100% with runnable examples

1.5 Non-Goals

To prevent scope creep, the following are explicitly declared non-goals for Sky v1 and v1.x. They may be revisited for v2 but must not influence v1 architecture decisions.

- A graphical or web-based user interface. Sky is and remains a CLI.
- A built-in code editor, file watcher, or language server. Sky composes with existing tools rather than replacing them.
- Cloud-hosted session storage or model serving. All persistence is local under ~/.sky/.
- A proprietary model fine-tune. Sky is model-agnostic; no provider receives preferential treatment beyond adapter completeness.
- Real-time collaboration or multi-user sessions. Sky is single-user, single-process by design.
- A plugin marketplace. Extension is via MCP (Model Context Protocol) servers and local scripts, not a curated store.

> **Architectural note.** The non-goal list is as important as the feature list. Every “no” above protects the project’s ability to ship a focused, reliable CLI. Proposals that violate a non-goal require an explicit architecture decision record (ADR) and maintainer consensus before they can be merged.

2. Architecture and Module Breakdown

Sky is structured as a layered, modular TypeScript application. The high-level architecture separates four concerns: **presentation** (the terminal UI), **orchestration** (the agent loop that drives the conversation), **capability** (the tools the agent can invoke), and **state** (sessions, configuration, and history). Each concern is a top-level directory with a typed public interface; cross-module communication happens exclusively through these interfaces, never through shared mutable state or direct filesystem access.

2.1 High-Level Architecture

The diagram below shows the runtime data flow of a single agent turn. The user types a prompt in the TUI; the TUI forwards it to the agent loop; the loop consults the session store for prior context, builds a message array, calls the configured LLM provider, parses the streamed response for tool calls, dispatches each tool call through the safety layer, executes the tool, appends the result to the session, and streams the assistant's prose back to the TUI. This cycle repeats until the assistant emits a final message with no tool calls.

Architecture diagram: layers and data flow (rendered separately).

2.2 Directory Structure

The repository follows a conventional Node.js layout with a strict separation between source (src/), tests (test/), and documentation (docs/). The src/ tree is organized by concern, not by feature, to keep the module boundaries visible at a glance.

2.3 Module Dependency Graph

Modules depend only inward and downward. The cli/ layer depends on everything; the llm/ and tools/ layers depend only on config/, logging/, and errors/; the config/, logging/, and errors/ layers have no internal dependencies. This ordering is enforced by an ESLint no-restricted-imports rule and checked in CI.

Module	Depends On	Depended On By	Public Interface
cli/	agent/, tui/, session/, config/	(entry point)	Commander command tree
agent/	llm/, tools/, safety/, session/, logging/	cli/	AgentLoop.run()
tui/	agent/, session/, logging/	cli/	<App /> React component
tools/	safety/, config/, logging/, errors/	agent/	Tool interface + registry
safety/	config/, logging/, errors/	tools/, agent/	Approver, Policy
session/	config/, logging/, errors/	agent/, tui/, cli/	SessionStore

Module	Depends On	Depended On By	Public Interface
config/	logging/, errors/	all	loadConfig()
llm/	config/, logging/, errors/	agent/	Provider interface
logging/	errors/	all	logger instance
errors/	(none)	all	ErrorCode enum + typed classes

2.4 Core Module Responsibilities

2.4.1 agent/ — Orchestration

The agent module is the brain of Sky. It owns the conversation loop: given a user message and a session, it produces a stream of assistant tokens and a sequence of tool calls. The loop is a generator function that yields events (text-delta, tool-call, tool-result, approval-request, error) as they happen, allowing the TUI to render them incrementally. This design decouples orchestration from presentation: the same loop drives the interactive TUI and the headless CI runner, the only difference being how events are consumed.

2.4.2 tui/ — Terminal User Interface

The TUI is a React application rendered into the terminal via Ink. It owns input handling (multi-line editing, history recall, slash commands), output rendering (streamed prose, tool-call banners, diff viewers), and the status bar (mode indicator, model name, token usage, session id). The TUI never calls LLM providers or tools directly; it consumes the agent loop's event stream and renders it. This separation is what allows the headless mode to reuse the entire agent module without linking against Ink.

The component tree is deliberately shallow: <App> owns the agent loop subscription and routes events to three children — <MessageStream> for assistant output, <MultiLineInput> for user typing, and <StatusBar> for persistent metadata. Modal overlays (diff approval, command palette, help) render conditionally on top.

2.4.3 tools/ — Capability Layer

Each tool is a self-contained module exporting a Tool implementation. The registry discovers tools at startup, validates their schemas against Zod, and exposes them to the agent loop by name. Tools are pure functions of their inputs: they do not read from the session, do not call the LLM, and do not render to the TUI. Their only side effects are on the filesystem and shell, and those side effects are mediated by the safety layer.

2.4.4 safety/ — Approval and Audit

The safety layer sits between the agent loop and the tool registry. Every tool call passes through Approver.request(), which consults a policy engine to decide whether the call is auto-approved, auto-denied, or requires interactive confirmation. The policy engine is configured by the user via ~/.sky/config.json and supports allowlist patterns (e.g. “read anything under src/”), denylist patterns (e.g. “never delete .env”), and shell-command classification rules. Every decision is written to an append-only audit log before the tool executes.

2.4.5 session/ — Persistence

The session module is the single source of truth for conversation state. It writes every message, tool call, and approval decision to a JSON file under ~/.sky/sessions/<id>.json using an atomic write strategy (write to temp, rename). Sessions are append-only during a run; compaction (summarizing old turns) happens only on explicit user request via sky compact <id> or automatically when a session exceeds a configurable size threshold.

The session schema is versioned. Each file includes a schemaVersion field; on load, the store runs a migration pipeline to bring old sessions up to the current version. This guarantees that sky resume works on sessions created by older Sky versions, which is critical for long-lived projects.

2.4.6 config/ — Configuration

The config module loads, validates, and provides access to user settings. The canonical source is ~/.sky/config.json, but values can be overridden by environment variables (prefix SKY_) and by per-project .skyrc files. The loader applies overrides in a fixed precedence order and validates the merged result against a Zod schema before returning it. Any validation failure aborts startup with a clear error message that names the offending field and the expected shape.

2.4.7 llm/ — Provider Abstraction

The LLM module defines a Provider interface that every adapter must implement. The interface exposes stream() (returns an async iterable of chunks), countTokens() (returns an integer for a given message array), and tokenLimits (the model’s context window and output cap). Adapters translate between this interface and the provider-specific SDK. This is the only module that imports openai, @anthropic-ai/sdk, or any other vendor package; the rest of Sky is vendor-agnostic.

2.4.8 logging/ — Observability

Logging uses pino for structured JSON output. The logger is created once at startup and injected into every module via a Logger interface. A redactor strips known secret patterns (API keys, bearer tokens) before any log line is written. Logs go to two sinks: ~/.sky/logs/sky.log (rotated daily, retained for 30 days) and stderr (when --verbose is passed).

2.4.9 errors/ — Error Taxonomy

Every error in Sky is an instance of SkyError, which carries a stable code (format SKY-E-XXXX), a human-readable message, an optional cause, and an optional retryable flag. The codes are partitioned by module prefix: 1xxx config, 2xxx agent/context, 3xxx tools, 4xxx session, 5xxx llm/provider, 6xxx safety, 7xxx tui, 8xxx cli. See Appendix B for the full catalog.

3. Tech Stack Rationale

Sky's dependency list is deliberately short. Every runtime dependency was chosen because it is the best-in-class option for its concern, has a maintained TypeScript type definition, and is permissively licensed. This section justifies each choice, lists the alternatives that were considered and rejected, and explains how the dependency fits into the module architecture defined in Section 2.

3.1 Runtime Dependencies

Dependency	Version	Module	Purpose	Alternatives Considered
TypeScript	5.4+	all	Type safety, interfaces, discriminated unions for events	Plain JS (rejected: lose safety); Flow (rejected: ecosystem shrinkage)
Node.js	20 LTS+	runtime	JavaScript runtime, fs, child_process, streams	Bun (rejected: stability for v1); Deno (rejected: npm compat friction)
Ink	5.x	tui/	React renderer for the terminal; composable TUI components	blessed (rejected: abandoned); Textual (rejected: Python, not TS)
React	18.x	tui/	Component model, hooks for state management	(required by Ink; not separately choosable)
Commander.js	12.x	cli/	Command tree, flag parsing, help generation	yargs (rejected: heavier); clipanion (rejected: smaller community)
Zod	3.x	all	Runtime schema validation for config, tool I/O, session files	io-ts (rejected: less ergonomic); joi (rejected: not TS-native)
openai	4.x	llm/	Official OpenAI SDK; also used by OpenRouter and Ollama adapters	fetch directly (rejected: lose streaming ergonomics)
@anthropic-ai/sdk	0.30+	llm/	Official Anthropic SDK for Claude models	fetch directly (rejected: lose streaming + retries)
pino	9.x	logging/	Structured JSON logger with low overhead	winston (rejected: slower); console.log (rejected: no structure)
p-retry	6.x	llm/	Promise retry with exponential backoff for transient failures	custom (rejected: well-tested lib exists)
execa	9.x	tools/shell	Child process execution with promise API and parsing	node:child_process (rejected: ergonomics); shelljs (rejected: global state)
simple-git	3.x	tools/git	Typed git command wrapper	execa + manual (rejected: reinventing wheel)
diff	5.x	safety/	Line-level and word-level diff generation for approvals	custom (rejected: correctness risk)
chalk	5.x	tui/	ANSI color for non-Ink surfaces (help text, headless output)	picocolors (acceptable alt; chalk chosen for ecosystem)

Dependency	Version	Module	Purpose	Alternatives Considered
gray-matter	4.x	docs/	Frontmatter parsing for ADR and doc files	custom (rejected: trivial but well-tested lib)
@modelcontextprotocol/sdk	0.5+	tools/mcp_proxy	MCP client for proxying to external tool servers	custom (rejected: protocol is still evolving; track official SDK)

3.2 Dev Dependencies

Development dependencies are pinned and audited weekly via npm audit. The list below is the minimum required to build, test, lint, and release Sky.

Dependency	Version	Purpose
vitest	1.x	Test runner (Jest-compatible API, native ESM, faster than Jest)
@vitest/coverage-v8	1.x	Coverage via V8 inspector (no instrumentation overhead)
esbuild	0.20+	Bundler for the CLI binary; produces a single-file CJS bundle
tsx	4.x	TypeScript execution for dev mode and scripts
eslint	9.x	Linter with typescript-eslint, import-access, no-restricted-imports
prettier	3.x	Code formatter (enforced via lint-staged + husky)
tsup	8.x	Alternative bundler config; used for producing ESM + CJS dual builds
type-doc	0.25+	API documentation generator from TSDoc comments
husky	9.x	Git hooks for pre-commit lint + tests
lint-staged	15.x	Run linters only on changed files
@changesets/cli	2.x	Changelog and release management
playwright	1.x	E2E tests for the TUI via headless terminal driver

3.3 Architectural Dependencies Between Modules

The dependency graph in Section 2.3 is enforced at build time. The mechanism is an ESLint rule that forbids imports outside a module's declared dependency list. Each module declares its allowed import roots in package.json under a custom sky.moduleImports field; the ESLint rule reads this field and fails the build on any violation. This prevents the most common form of architectural decay: a tool importing from the TUI, or the agent importing a provider-specific SDK.

```
// package.json (excerpt)
{
  "sky": {
    "moduleImports": {
      "src/cli": ["src/*"],
      "src/agent": ["src/llm", "src/tools", "src/safety",
                    "src/session", "src/logging", "src/errors"],
      "src/tui": ["src/agent", "src/session", "src/logging",
                    "src/errors"],
      "src/tools": ["src/safety", "src/config", "src/logging",
                    "src/errors"],
      "src/safety": ["src/config", "src/logging", "src/errors"],
      "src/session": ["src/config", "src/logging", "src/errors"],
      "src/config": ["src/logging", "src/errors"],
      "src/llm": ["src/config", "src/logging", "src/errors"],
      "src/logging": ["src/errors"],
      "src/errors": []
    }
  }
}
```

Figure 3.1 — Per-module import allowlist. Enforced by eslint-plugin-import-access.

3.4 Why TypeScript (Not JavaScript, Not Python)

TypeScript was chosen over plain JavaScript because Sky's surface area is large enough that runtime errors from shape mismatches would dominate the bug tracker. The agent loop yields a discriminated union of event types; tool inputs and outputs are validated against Zod schemas; the session file format is a typed structure. Without compile-time checking, these invariants would rely solely on runtime validation, which catches bugs only when the relevant code path executes. TypeScript catches them at tsc time, which is seconds, not minutes.

Python was considered and rejected for two reasons. First, the target audience (backend engineers, DevOps, open-source maintainers) is overwhelmingly Node-fluent; requiring a Python install would shrink the contributor pool. Second, Ink and the broader React ecosystem give Sky a component model for the TUI that has no peer in Python. Textual is excellent, but it does not match Ink's composability with hooks, suspense, or the wider React knowledge base.

3.5 Why Ink (Not blessed, Not a Custom TUI)

Ink is a React renderer for the terminal. It lets the TUI be expressed as a tree of function components, with state managed via hooks and effects. This is the same mental model millions of frontend engineers already know, which lowers the contribution barrier. Ink also handles the hard parts of terminal rendering — cursor positioning, ANSI escape codes, layout reflow on resize — that a custom TUI would have to solve from scratch.

blessed was the previous-generation choice for terminal UIs in Node. It has not been actively maintained since 2017, has known rendering bugs on modern terminals, and its widget set is opinionated toward full-screen applications rather than the inline, streaming layout Sky needs. A custom renderer was estimated at 6–8 weeks

of work to reach feature parity with Ink's existing flexbox implementation, with no compensating benefit.

3.6 Why Commander.js (Not yargs, Not clipanion)

Commander.js is the most widely used CLI framework in the Node ecosystem. It has first-class TypeScript support, a stable API, and excellent documentation. Its command tree model maps directly onto Sky's command surface (sky agent, sky plan, sky resume, etc.), and its help text generation is good enough that the default output is shippable with minimal customization.

yargs was rejected because its configuration-heavy style (large option objects, implicit coercion rules) makes the command definitions harder to read and refactor as the flag set grows. clipanion is technically excellent and used by the Yarn team, but its community is an order of magnitude smaller, and the contributor onboarding cost outweighs its marginal type-safety advantages over Commander.

3.7 Why Zod (Not io-ts, Not joi)

Zod is the runtime validation library used across Sky for config, tool I/O, and session file parsing. It was chosen because its schema syntax reads like TypeScript type definitions, it infers static types from schemas (so the schema and the type are never out of sync), and it produces detailed error messages that name the offending field. Sky's error UX depends heavily on the last point: when a config file is malformed, the user needs to see exactly which field is wrong, not a generic "validation failed."

io-ts was rejected because its schema definition style (functional combinators) is less readable for engineers unfamiliar with fp-ts. joi was rejected because it is not TypeScript-native (types are inferred separately, leading to drift), and its error formatting is less structured than Zod's.

3.8 Why Official LLM SDKs (Not Raw fetch)

Sky uses the official openai and @anthropic-ai/sdk packages rather than calling provider REST endpoints directly. The official SDKs handle streaming parsing (Server-Sent Events for OpenAI, message-stream for Anthropic), automatic retries on 429 and 5xx, request signing, and beta-feature flag plumbing. Reimplementing these in-house would be a steady source of bugs as providers evolve their APIs. The cost is a heavier dependency tree, but the SDKs are tree-shakeable and only their used code paths end up in the bundle.

For Ollama and OpenRouter, the OpenAI SDK is reused because both providers expose OpenAI-compatible endpoints. This is a pragmatic decision: it reduces the adapter surface, and it means features like streaming and tool-calling work without provider-specific code paths. The Ollama adapter overrides the base URL and disables features Ollama does not support (e.g. log-probs); the OpenRouter adapter adds the HTTP-Referer header required by the OpenRouter policy.

3.9 Express Integration (Optional HTTP API)

Although Sky is primarily a CLI, the specification includes an optional embedded HTTP server for programmatic access. This is enabled by running sky --serve <port>, which starts an Express.js server exposing a small REST API. The server is intended for integration with editor extensions, dashboards, or CI orchestrators that need to drive Sky programmatically without shelling out to the CLI. The server is opt-in; when not enabled, Express is not imported and adds zero runtime cost.

The Express integration is deliberately minimal: three endpoints, NDJSON streaming for events, no authentication (it binds to localhost only), no WebSocket (streaming via chunked HTTP is sufficient). If a richer API is needed in the future, it will be added under /v2/ to preserve backward compatibility.

4. CLI Command Reference

Sky's command surface is intentionally small. There is one top-level command per primary user intent, plus a handful of utility commands for session and configuration management. Every command supports --help with runnable examples, --version, and the standard --verbose / --quiet flags. Exit codes follow the BSD convention: 0 for success, 1 for runtime error, 2 for usage error, 64+ for specific error classes (see Appendix B).

4.1 Command Catalog

Command	Mode	Description	Mutates Workspace
sky	agent	Default: start an interactive agent session in the current directory	Yes (with approval)
sky agent	agent	Explicit form of the default; same as bare sky	Yes (with approval)
sky plan	plan	Plan-first mode: agent clarifies and designs before any change	No (until approved)
sky ask	ask	Read-only Q&A; no tools, no file mutation	No
sky resume [id]	—	Resume an existing session by id or interactively	Depends on resumed mode
sky ls	—	List sessions for the current directory	No
sky config	—	Open config in $EDITOR, or sky config get/set <key>	No (config only)
sky init	—	Create ~/.sky/config.json with defaults and prompt for API key	No (config only)
sky mcp	—	Manage MCP server registrations: add/list/remove/test	No (config only)
sky compact [id]	—	Summarize old turns in a session to reclaim context	No
sky --serve [port]	agent	Start the optional HTTP API server instead of the TUI	Yes (via API)
sky --version	—	Print version and exit	No
sky --help	—	Print top-level help and exit	No

4.2 Global Flags

These flags apply to every command. They must appear before any subcommand arguments, in line with Commander.js convention.

Flag	Type	Default	Description
--model, -m	string	from config	Override the model for this run (e.g. gpt-4o, claude-3-5-sonnet)

Flag	Type	Default	Description
--provider, -p	string	from config	Override the LLM provider (openai, anthropic, ollama, openrouter)
--yolo	boolean	false	Auto-approve every tool call. Implies --force. Use only in CI.
--force	boolean	false	Skip interactive confirmations (file overwrite, etc.) but still respect denylist.
--cwd	path	process.cwd()	Run as if started in this directory. Useful for scripting.
--session, -s	string	auto-generated	Resume a specific session by id; fail if it does not exist.
--config, -c	path	~/.sky/config.json	Path to an alternative config file.
--verbose	boolean	false	Emit debug logs to stderr; sets log level to debug.
--quiet	boolean	false	Suppress all non-error stderr output.
--no-color	boolean	auto	Disable ANSI color. Default respects NO_COLOR env var.
--json	boolean	false	Output events as NDJSON on stdout. Implied when --serve is used.

4.3 The sky Command (Default Agent Mode)

Invoking sky with no arguments (or with sky agent) starts an interactive agent session in the current directory. The agent reads the workspace, listens for user input, and executes tool calls as needed, subject to the approval policy in effect. This is the primary command and the one most users will run.

```
# Start an agent in the current directory
$ sky

# Start with a specific model and provider
$ sky --provider anthropic --model claude-3-5-sonnet

# Start with an initial prompt (non-interactive if --yolo is also set)
$ sky "refactor src/auth to use async/await throughout"

# Resume the most recent session in this directory
$ sky --session latest

# Run headless in CI: auto-approve, stream JSON events
$ sky --yolo --json "fix lint errors in src/" < prompts.txt
```

Figure 4.1 — Common invocations of the default sky command.

4.4 The sky plan Command

Plan mode is for tasks where the user wants a design before any code is written. In this mode, the agent does not execute tools; instead, it asks clarifying questions, drafts a plan, and presents it for review. Once the user approves, the agent switches to agent mode and executes the plan. This two-phase flow is valuable for large refactors, unfamiliar codebases, and any task where the cost of a wrong guess is high.

4.5 The sky ask Command

Ask mode is read-only. The agent has no tools; it can only read files that the user passes in (via @file mentions) and answer questions about them. This is the safest mode and the one recommended for new team members exploring an unfamiliar codebase. It is also useful for quick questions that do not warrant starting a full agent session.

```
$ sky ask "what does src/auth/token.ts do?"

◆ Reading @src/auth/token.ts
◆ Reading @src/auth/types.ts (referenced import)

`src/auth/token.ts` is responsible for issuing, signing, and verifying
JWT tokens for the API. It exports three functions:

- `issueToken(user)` — creates a signed JWT with a 1-hour expiry
- `verifyToken(token)` — validates signature and expiry, returns the payload
- `refreshToken(token)` — exchanges an expiring token for a fresh one

The signing key is read from `process.env.JWT_SECRET` at module load time.
If the env var is missing, the module throws on first use rather than at
import time, which is a deliberate choice to allow test mocks.
```

4.6 The sky resume Command

Resume reloads a saved session and continues it. With no argument, it shows an interactive picker of recent sessions for the current directory. With a session id, it loads that session directly. Resumed sessions retain their entire message history, tool results, and approval decisions, so the agent has full context of what was already done.

```
# Interactive picker
$ sky resume

# Resume a specific session
$ sky resume abc123

# Resume and immediately continue with a follow-up prompt
$ sky resume abc123 "now add tests for the new endpoint"

# Resume in view-only mode (cannot send new messages, just scroll history)
$ sky resume abc123 --view
```

4.7 The sky ls Command

Lists sessions, optionally filtered by directory, date range, or status. The output is a table by default; with --json it is a JSON array suitable for scripting.

```
$ sky ls

ID MODE STARTED MESSAGES LAST ACTIVITY CWD
abc123 agent 2026-07-12 09:14:32 42 2026-07-12 10:02:11 ~/projects/api
def456 plan 2026-07-12 08:50:01 18 2026-07-12 09:05:44 ~/projects/api
ghi789 ask 2026-07-11 17:22:10 7 2026-07-11 17:31:55 ~/projects/api

$ sky ls --cwd ~/projects/api --since 7d --json | jq '.[] | .id'
"abc123"
"def456"
"ghi789"
```

4.8 The sky config Command

Manages the configuration file. With no subcommand, it opens the config in $EDITOR. With get / set / unset subcommands, it reads or writes individual keys without touching the rest of the file. All mutations validate against the Zod schema before writing.

```
# Open config in editor
$ sky config

# Read a single key
$ sky config get providers.openai.model
gpt-4o

# Set a key (validates against schema)
$ sky config set providers.anthropic.apiKeyEnv ANTHROPIC_API_KEY

# List all keys with current values
$ sky config list

# Validate config without modifying it
$ sky config validate
✓ Config is valid (47 keys, 3 providers configured)
```

4.9 The sky mcp Command

Manages registrations for Model Context Protocol (MCP) servers. MCP is an open protocol that allows external tool servers (e.g. a database query server, a Jira client, a custom internal API) to expose their capabilities to Sky. The sky mcp command lets the user add, list, remove, and test these registrations without editing JSON by hand.

```
# Register an MCP server
$ sky mcp add my-db \
    --command "node" \
    --args "/usr/local/bin/mcp-pg-server" \
    --env PG_CONNSTRING="postgres://localhost/mydb"

# List registered servers
$ sky mcp list
NAME COMMAND STATUS
my-db node mcp-pg-server ● healthy
jira npx @mcp/jira-server ● healthy
slack npx @mcp/slack-server ○ untested

# Test a server (lists the tools it exposes)
$ sky mcp test my-db
✓ Connected in 312ms
Tools exposed:
  - query(sql: string) -> rows
  - list_tables() -> string[]
  - describe_table(name: string) -> schema

# Remove a server
$ sky mcp remove slack
```

4.10 The sky init Command

Creates ~/.sky/config.json with sensible defaults and walks the user through choosing a provider and entering an API key. The API key is stored in the system keychain (via keytar) when available, falling back to an env var reference in the config. sky init is idempotent: running it on an existing config prompts before overwriting.

4.11 Exit Codes

Code	Meaning	Example Trigger
0	Success	Agent completed normally or session saved
1	Runtime error	Unhandled exception, tool execution failure
2	Usage error	Invalid flag, missing required argument
64	Config error	Malformed config, missing API key
65	Session error	Corrupt session file, unknown session id
66	Provider error	LLM provider returned 4xx/5xx, rate limit
67	Safety denial	User denied a required approval; agent aborted
68	Context error	Token budget exceeded, compaction required
70	Internal error	Invariant violation; please file a bug
130	Interrupted	SIGINT (Ctrl-C) received; session auto-saved

> **Convention.** Exit codes 64–78 follow the BSD sysexits.h range for non-runtime failures. CI scripts should treat any non-zero exit as a failure, but scripts that need to distinguish “user denied” from “provider down” can inspect the specific code.

5. UI/UX Specification

Sky's terminal user interface is the primary surface through which users interact with the agent. It is designed to be legible at a glance, responsive under streaming output, and unambiguous about what the agent is doing at every moment. The visual language is sparse: a small set of glyphs, a single accent color, and generous whitespace. This section specifies every visual element, its rendering rules, and the interaction model that governs it.

5.1 Visual Language

The TUI uses a single glyph — the hex character ⬢ (U+2B22, "Black Hexagon") — as the universal status indicator. The hex appears before every status line, colored according to the agent's current activity. This convention is borrowed from developer tooling that uses geometric shapes as glanceable status markers; the hex was chosen over the more common circle or square because its six-fold symmetry makes it visually distinct from typical terminal text without being noisy.

Glyph	Color	State	Meaning
⬢	gray	Idle	Agent is waiting for user input
⬢	yellow	Thinking	Agent is processing; no tokens streamed yet
⬢	cyan	Responding	Tokens are streaming from the provider
⬢	blue	Editing	A file write/edit tool is executing
⬢	magenta	Searching	A search or read tool is executing
⬢	red	Error	An error occurred; awaiting user action
⬢	green	Done	Turn complete; ready for next input
⬢	bright yellow	Approval	Awaiting user approval for a tool call

5.2 Layout

The terminal is divided into three regions, top to bottom: the message stream (scrolling history of user and assistant turns), the status bar (a single line of persistent metadata), and the input area (multi-line editor). When an approval is requested, a modal overlay covers the input area; the message stream remains visible above so the user has context for the decision.

Figure 5.1 — Default TUI layout during an agent turn.

5.3 Status Bar

The status bar is a single line at the bottom of the message stream, above the input area. It persists across turns and conveys the four pieces of information the user most often wants at a glance: the current mode, the active model, the token usage, and the session id. The bar is rendered with a subtle background to separate it visually from the message stream without dominating the screen.

Segment	Content	Color	Width
Mode indicator	Hex glyph + mode name (agent/plan/ask)	mode-colored	~12 chars
Model	Provider:model (e.g. anthropic:claude-3-5-sonnet)	gray	~32 chars
Token usage	used / limit with color cue (green<90%, yellow<95%, red≥95%)	contextual	~22 chars
Session id	First 5 chars of session id, clickable to copy full id	accent	~8 chars

5.4 Multi-Line Input

The input area supports multi-line editing. Enter inserts a newline unless the cursor is on the last line and the line is non-empty, in which case Enter submits the input. This follows the convention used by Slack, Discord, and most modern chat UIs, and is the most common request from users coming from single-line CLI tools. The submit-on-Enter behavior can be inverted via config (tui.submitOnEnter: false) for users who prefer the traditional shell behavior.

Key	Action
Enter (last line, non-empty)	Submit input to agent
Enter (otherwise)	Insert newline
Shift+Enter	Insert newline (always)
Ctrl+J	Insert newline (always)
Up / Down	Move cursor across lines; recall history when on first/last line
Ctrl+Up / Ctrl+Down	Recall history regardless of cursor position
Tab	Autocomplete @mention, /command, or file path
Shift+Tab	Insert literal Tab character
Ctrl+C	Cancel current agent turn; second press exits Sky
Ctrl+D	Exit Sky (EOF)
Ctrl+L	Clear screen (preserves session history)
Ctrl+R	Reverse history search
Esc	Cancel current input or close modal overlay

5.5 Slash Commands

Typing a line beginning with / in the input area opens the slash command palette. Slash commands are shortcuts that do not require a full agent turn; they execute immediately and synchronously.

Command	Description
/help	Show help overlay with keybindings and slash command list
/mode [agent|plan|ask]	Switch mode for the current session
/model [name]	Switch model for the current session
/compact	Run session compaction (summarize old turns)
/clear	Clear the message stream visually (preserves session)
/undo	Undo the last tool call (if reversible)
/diff	Show all uncommitted changes the agent made in this session
/save [name]	Save the current session with a friendly name
/export [format]	Export session as markdown, json, or text

Command	Description
/cost	Show token and estimated cost breakdown for this session
/exit	Exit Sky (saves session)

5.6 Diff Approval Prompt

When the agent proposes a file mutation (write or edit), the TUI renders a diff viewer inline and prompts the user to approve, reject, or edit the change before it is applied. The diff viewer uses unified diff format with color: green for additions, red for deletions, gray for context. Long diffs are paginated; the user can scroll with j/k or arrow keys before deciding.

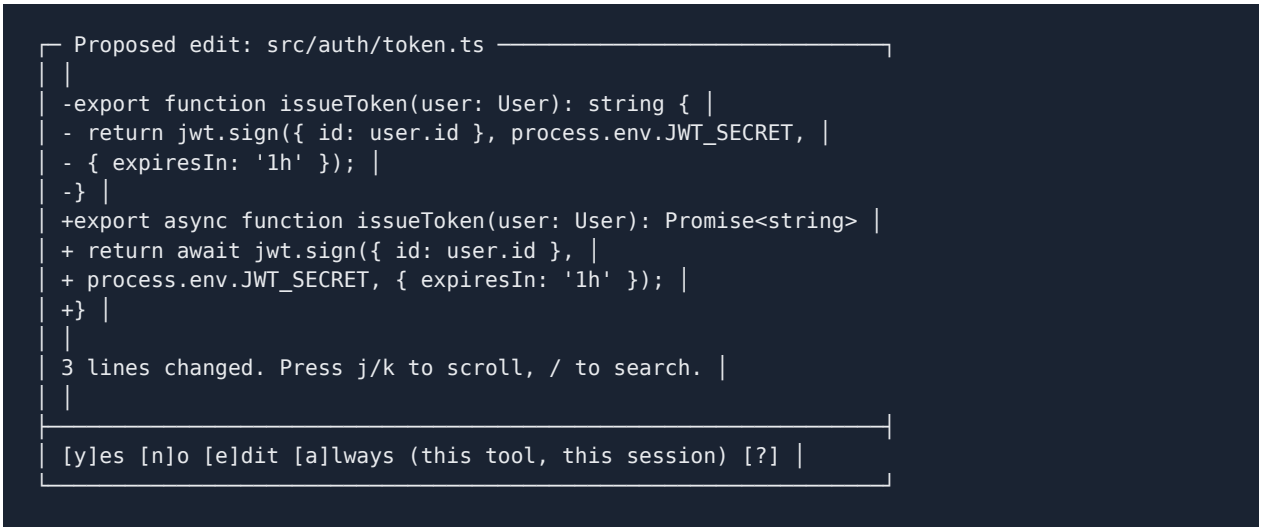

Figure 5.2 — Diff approval prompt. The user can approve, reject, edit in $EDITOR, or auto-approve future calls of this tool.

5.7 Color Palette and Accessibility

Sky uses a small, opinionated color palette. All colors are derived from the terminal's 256-color palette (not truecolor) to maximize compatibility. The palette respects NO_COLOR and FORCE_COLOR environment variables, and the --no-color flag overrides both. When color is disabled, all information conveyed by color is also conveyed by glyph or text (e.g. the hex indicator shows the mode even in monochrome).

Semantic Name	Color	Used For
primary	bright white	Default body text
muted	gray	Secondary text, metadata, timestamps
accent	cyan	Active element, links, session id
success	green	Completion, approval granted, ok status
warning	yellow	Caution, near-limit token usage

Semantic Name	Color	Used For
error	red	Errors, denials, destructive actions
info	blue	Informational, agent activity
planning	magenta	Search/read activity, plan mode emphasis

5.8 Theme Customization

Users can override any color or glyph via the tui.theme section of ~/.sky/config.json. The override file maps semantic names to ANSI color codes (numeric or named). Invalid overrides are reported at startup with the offending key and value, and the user is offered the choice to reset to defaults or continue with the invalid override disabled.

```
{
  "tui": {
    "theme": {
      "colors": {
        "accent": "cyan",
        "success": "green",
        "error": "red",
        "warning": "yellow",
        "info": "blue",
        "planning": "magenta"
      },
      "glyphs": {
        "indicator": "\u2B22",
        "bullet": "\u2022",
        "arrow": "\u2192"
      },
      "layout": {
        "submitOnEnter": true,
        "showTokenBar": true,
        "compactMode": false
      }
    }
  }
}
```

Figure 5.3 — Theme override structure in config.json.

5.9 Responsive Behavior

The TUI reflows on terminal resize. The message stream re-wraps to the new width, the status bar re-segments, and the input area adjusts its visible line count. A resize during a streaming turn does not interrupt the stream; the next chunk renders at the new width. Below 80 columns, Sky switches to compactMode (configurable): the status bar collapses to a single segment (mode + token count), diffs switch to a narrower format, and the message stream uses a 2-space indent instead of a 4-space one. Below 60 columns, an informational banner suggests widening the terminal for a better experience.

5.10 Headless Mode Rendering

When Sky runs with --json (typical in CI), the TUI is not initialized. Instead, events from the agent loop are serialized as newline-delimited JSON and written to stdout. Each event has a type field and a payload field; consumers can parse the stream incrementally. This is the same event stream the TUI consumes internally; the headless mode simply serializes it instead of rendering it.

6. Tool Implementation Details

Tools are the agent's hands. Each tool is a self-contained module that exposes a typed interface, validates its inputs against a Zod schema, executes against the local filesystem or shell, and returns a structured result. This section documents every tool's interface, behavior, error handling, and safety constraints in enough detail for an engineer to implement or extend it without ambiguity.

6.1 Tool Interface Contract

Every tool implements the Tool interface defined in src/tools/registry.ts. The interface is intentionally minimal: four fields describing the tool, and one method that executes it. The requiresApproval predicate lets a tool express that some inputs are safe (e.g. reading a file under src/) while others require approval (e.g. reading .env); this keeps the safety policy granular and contextual.

6.2 The read Tool

Reads the contents of a file at the given path, returning it as a string. The path is resolved relative to the session's cwd. Binary files are detected (via null-byte heuristic) and returned as a placeholder with file metadata rather than as raw bytes. Files larger than a configurable threshold (default 256 KB) are truncated with a notice; the agent can request specific line ranges via the offset and limit parameters.

6.3 The write Tool

Writes content to a file, creating it if it does not exist and overwriting it if it does. Write is the most dangerous tool because it can destroy existing content; it therefore always requires approval unless the user has explicitly added the target path to an allowlist in config. The tool refuses to write outside the session cwd unless allowOutsideCwd is set in config (default false). Before writing, the tool reads the current content (if any) and passes both old and new content to the safety layer, which generates a diff for the approval prompt.

6.4 The edit Tool

Edits a file by replacing a specific old string with a new string. Edit is preferred over write for targeted changes because it produces a smaller diff and fails loudly if the old string is not found (catching drift between the agent's mental model and the file's actual content). The tool supports an occurrences field ('all' or a positive integer) to control multi-match behavior; the default is to fail if the old string appears more than once, forcing the agent to disambiguate.

6.5 The search Tool

Searches file contents using ripgrep (preferred) or grep as a fallback. The tool accepts a regex pattern, a path (file or directory), and optional filters (glob, case sensitivity, max results). Results are returned as a list of matches with file path, line number, and line content. The tool is read-only and never requires approval when searching within the workspace cwd; searches outside cwd require approval.

6.6 The shell Tool

Executes an arbitrary shell command in the session cwd. This is the highest-risk tool because it can do anything the user can do: delete files, push to remotes, install packages, exfiltrate data. The shell tool therefore has the strictest safety policy: every invocation requires approval (even in --yolo mode, unless the command matches an explicit allowlist pattern), the command is classified before execution, and a denylist of destructive patterns is always enforced regardless of user approval.

6.7 The git Tool

A typed wrapper around common git operations. The git tool exists because shelling out to git via the shell tool would lose structure (the agent would have to parse git's text output) and would make safety policy harder to enforce (every git subcommand has different risk). By exposing git as a first-class tool with typed sub-actions, Sky can apply targeted approval rules: status and log are auto-approved, commit requires approval, push --force is always denied.

6.8 The mcp_proxy Tool

A meta-tool that forwards calls to external MCP (Model Context Protocol) servers. When the user registers an MCP server via sky mcp add, the server's tools are discovered at startup and each is wrapped in a synthetic mcp.<server>.<tool> tool. The agent can call these tools exactly like built-in ones; the proxy forwards the call over stdio to the MCP server, awaits the result, and returns it. The approval policy for MCP tools is configurable per server: auto (approve all), manual (approve each), or deny (never call). The default is manual.

6.9 Tool Error Handling Summary

Every tool returns a ToolResult with ok: boolean. Failures are categorized into recoverable (the agent can retry with different inputs) and unrecoverable (the agent should report to the user and stop). The table below maps the major error codes to their categories and retry behavior.

Code	Tool	Meaning	Recoverable	Agent Behavior
SKY-E-3001	any	Unknown tool name	No	Report and stop

Code	Tool	Meaning	Recoverable	Agent Behavior
SKY-E-3002	any	Invalid input (failed Zod)	Yes	Retry with corrected input
SKY-E-3003	any	Tool returned invalid output	No	Report and stop (bug)
SKY-E-3010	write	Path outside cwd	Yes	Retry with corrected path
SKY-E-3020	edit	oldText not found	Yes	Re-read file, retry
SKY-E-3021	edit	oldText ambiguous	Yes	Retry with more context
SKY-E-3030	search	ripgrep failed	No	Report and stop
SKY-E-3040	shell	Command denied (denylist)	No	Report and stop
SKY-E-3050	git	Force push denied	No	Report and stop
SKY-E-3060	mcp	MCP server in deny mode	No	Report and stop
SKY-E-3999	any	Unexpected error	No	Report and stop (bug)

7. Session and Configuration Management

Sky persists two kinds of state on disk: **sessions** (the conversation history, tool calls, and approval decisions for each agent run) and **configuration** (user preferences, provider credentials, tool policies). Both live under ~/.sky/, both are JSON files, and both are validated against Zod schemas on every read. This section specifies the on-disk format, the load/save logic, and the migration strategy for schema evolution.

7.1 The ~/.sky/ Directory

7.2 Session File Format

Each session is a single JSON file under ~/.sky/sessions/. The file is written atomically (temp file + rename) on every state change to guarantee crash consistency. The schema is versioned; the loader runs a migration pipeline on read if schemaVersion is older than the current version.

7.3 Session Lifecycle

A session moves through four states: active (currently being used), paused (saved but not currently active), compacted (old turns have been summarized to reclaim context), and archived (user has marked it as no longer relevant; retained for audit but hidden from default lists). The lifecycle is managed by the session store; the user can move between states via slash commands (/compact, /archive) or the CLI (sky compact <id>).

Transition	Trigger	Effect
(new) → active	sky / sky agent / sky plan / sky ask	Create session file, set status=active
active → paused	User exits (Ctrl+D, /exit, SIGTERM)	Flush pending writes, set status=paused
paused → active	sky resume <id>	Load file, run migrations, set status=active
paused → compacted	sky compact <id> or auto-threshold	Summarize old turns, replace with summary message, set status=compacted
compacted → active	sky resume <id>	Same as paused→active; summary message is preserved
any → archived	sky archive <id> or /archive	Set status=archived; hidden from default sky ls
archived → paused	sky unarchive <id>	Restore visibility

7.4 Session Index

To make sky ls fast even with thousands of sessions, Sky maintains an append-only index at ~/.sky/sessions.index. Each line is a JSON object with the session id, cwd, started, lastActivity, mode, and message count. The index is rebuilt from scratch on startup if it is missing or if its checksum does not match the sessions directory. Writes to the index are atomic (append + fsync); a corrupt last line is detected on read and truncated.

```
{"id":"abc123","cwd":"/home/alice/projects/api","started":"2026-07-12T09:14:32Z",
  "lastActivity":"2026-07-12T10:02:11Z","mode":"agent","messages":42}
{"id":"def456","cwd":"/home/alice/projects/api","started":"2026-07-12T08:50:01Z",
  "lastActivity":"2026-07-12T09:05:44Z","mode":"plan","messages":18}
```

Figure 7.3 — sessions.index format. One JSON object per line.

7.5 Configuration Schema

The configuration file at ~/.sky/config.json is the source of truth for all user preferences. It is validated against a Zod schema on every load; validation failures produce a structured error listing every offending field. The schema is exported to ~/.sky/config.schema.json on first run so that editors with JSON schema support can offer autocomplete and validation.

7.6 Configuration Precedence

Configuration values can come from multiple sources. The loader merges them in a fixed precedence order: later sources override earlier ones. After merging, the result is validated against the schema; if validation fails, the error message names the offending source so the user knows where to fix it.

Precedence	Source	Example	Notes
1 (lowest)	Schema defaults	tui.theme.colors.accent = "cyan"	Defined in schema.ts
2	~/.sky/config.json	"defaultModel": "gpt-4o"	Canonical user config
3	.skyrc in cwd	"defaultModel": "gpt-4o-mini"	Per-project override; not committed
4	Environment variables (SKY_*)	SKY_DEFAULT_MODEL=gpt-4o	Underscore-separated path; arrays via JSON
5	CLI flags	--model gpt-4o	Highest precedence; always wins

7.7 Secret Resolution

API keys are never written in plaintext to config.json by default. Instead, the config stores the *name* of the env var or keychain entry that holds the key, and the secrets module resolves it at runtime. This prevents accidental commit of credentials and works naturally across machines (each machine sets its own env var). The resolution order is:

1. If providers.X.apiKey is set in config, use it (legacy/explicit; logged as a warning at startup).
2. Else if providers.X.apiKeyEnv is set, read the env var of that name.
3. Else if the system keychain has an entry for sky/providers/X, use it.
4. Else if SKY_PROVIDERS_X_API_KEY env var is set, use it.
5. Else fail with SKY-E-1002: No API key configured for provider X.

7.8 Schema Migration

When the session or config schema changes, a migration is shipped. Migrations are pure functions: given an old-version object, they return a new-version object. They never mutate in place, never read from disk, and never fail silently — an unmigratable file is reported to the user with the original preserved as a backup. Migrations run automatically on load; the user never has to run a manual upgrade.

8. LLM Integration

Sky's LLM integration is built around a single Provider interface that every adapter must implement. The interface is small enough to specify in 30 lines of TypeScript, but rich enough to support streaming, tool calling, token counting, and cost tracking. This section documents the interface, each adapter's specifics, context window management, and the retry strategy for transient failures.

8.1 The Provider Interface

8.2 Adapter Catalog

Adapter	Provider	SDK	Streaming	Tool Calling	Token Counter
openai.ts	OpenAI	openai	SSE	function calling	tiktoken (cl100k for gpt-4o)
anthropic.ts	Anthropic	@anthropic-ai/sdk	message-stream	tool_use blocks	@anthropic-ai/tokenizer
ollama.ts	Ollama (local)	openai (compat)	SSE	function calling	Heuristic (4 chars ≈ 1 token)
openrouter.ts	OpenRouter	openai (compat)	SSE	function calling	tiktoken (approximate)

8.3 OpenAI Adapter

The OpenAI adapter uses the official openai npm package. It supports all OpenAI chat models (gpt-4o, gpt-4-turbo, gpt-3.5-turbo, o1, etc.) and any OpenAI-compatible endpoint when baseUrl is overridden. Tool calling uses OpenAI's function calling API; the adapter translates Sky's tool definitions into the OpenAI function schema format and parses the tool_calls field in the response.

8.4 Anthropic Adapter

The Anthropic adapter uses @anthropic-ai/sdk. It supports Claude 3.5 Sonnet, Claude 3 Opus, and Claude 3 Haiku. The adapter translates between Sky's message format and Anthropic's "messages" API, which differs from OpenAI's in several ways: system prompts are passed out-of-band, tool calls are returned as tool_use content blocks rather than a top-level field, and tool results are sent as tool_result blocks inside a user message.

8.5 Ollama Adapter

The Ollama adapter talks to a local Ollama server via its OpenAI-compatible endpoint (/v1/chat/completions). This is preferred over the native Ollama API because it reuses the OpenAI adapter's streaming and tool-calling logic. The adapter overrides the base URL to http://localhost:11434/v1 (configurable), disables features Ollama does not support (log-probs, stream_options.include_usage), and provides a heuristic token counter (4 characters ≈ 1 token) for models without a published tokenizer.

8.6 Context Window Management

The agent loop must keep the total token count of the message array under the model's context window. This is the responsibility of buildContext() in src/agent/context.ts. The function assembles the message array in priority order, counts tokens, and trims from the lowest-priority end until the total fits within the budget. The budget is contextWindow - maxOutput - safetyMargin, where the safety margin (default 2,048 tokens) accounts for tokenizer drift.

Priority	Content	Trimming Behavior
1 (highest)	System prompt (mode instructions, tool descriptions)	Never trimmed
2	Current user message	Never trimmed
3	Last N assistant turns (N configurable, default 6)	Trim oldest first
4	Tool results from current turn	Trim largest first; keep first 50 lines
5	Older assistant turns beyond N	Trim oldest first
6 (lowest)	Old tool results	Replace with stub: "[tool result trimmed] (X bytes)"
7 (lowest)	Compaction summary (if present)	Never trimmed (already the trimmings)

8.7 Retry Strategy

Transient failures (network errors, 429 Too Many Requests, 503 Service Unavailable) are retried with exponential backoff and jitter. The retry logic lives in src/llm/retry.ts and wraps every provider call. The maximum number of retries, the base delay, and the maximum delay are configurable per provider; defaults are 4 retries, 1s base, 30s max.

8.8 Provider Fallback

For critical workflows (e.g. CI runs), Sky supports provider fallback: if the primary provider fails all retries, the request is retried with the configured fallback provider. Fallback is configured in config.json under providers.X.fallback and is off by default. When fallback is used, the session records both providers and the failure that triggered the switch, so the audit trail is complete.

```
{
  "defaultProvider": "openai",
  "defaultModel": "gpt-4o",
  "providers": {
    "openai": {
      "apiKeyEnv": "OPENAI_API_KEY",
      "defaultModel": "gpt-4o",
      "fallback": {
        "provider": "anthropic",
        "model": "claude-3-5-sonnet",
        "triggerAfter": 4
      }
    },
    "anthropic": {
      "apiKeyEnv": "ANTHROPIC_API_KEY",
      "defaultModel": "claude-3-5-sonnet"
    }
  }
}
```

Figure 8.1 — Provider fallback configuration. After 4 retries on OpenAI, Sky falls back to Anthropic.

8.9 Cost Tracking and Budgets

Every provider call accumulates token usage in the session. The session's tokenUsage field is the running total; /cost displays the breakdown. A configurable budget cap (session.budgetUsd, default unlimited) aborts the session when the estimated cost exceeds the cap. This is useful for shared CI runners where a runaway agent could otherwise incur significant cost.

Model	Input ($/Mtok)	Output ($/Mtok)	Typical Cost per Session
gpt-4o	$2.50	$10.00	$0.03–$0.15
gpt-4o-mini	$0.15	$0.60	$0.002–$0.01
claude-3-5-sonnet	$3.00	$15.00	$0.04–$0.20
claude-3-haiku	$0.25	$1.25	$0.003–$0.02
ollama (local)	$0.00	$0.00	$0.00 (compute only)

9. Safety and Approval Workflows

Safety is the non-negotiable core of Sky's design. Every action that mutates state — writing a file, running a shell command, pushing to a remote — passes through a layered approval system that classifies the action, consults the user's policy, prompts for confirmation when required, and logs the decision to an append-only audit trail. This section specifies the policy engine, the interactive approval flow, the headless (auto-approval) flow, and the audit log format.

9.1 Safety Layer Architecture

The safety layer sits between the agent loop and the tool registry. Every tool call passes through three stages: **classification** (is this call allowed, denied, or requiring approval?), **authorization** (if required, prompt the user or consult the auto-approval policy), and **audit** (log the decision and outcome). The tool only executes if classification returns allow or authorization returns granted.

9.2 Policy Engine

The policy engine combines static rules (from config) with dynamic state (session allowlists added via "always" approvals). The classification logic evaluates rules in a fixed order: denylist first (always wins), then session allowlist, then config allowlist, then tool's own requiresApproval predicate. If none of these returns a definitive answer, the default is prompt.

9.3 Diff Generation and Approval

When the agent proposes a file write or edit, the safety layer generates a unified diff between the current and proposed content. The diff is displayed in the TUI with color (green for additions, red for deletions) and the user is prompted to approve, reject, or edit. The diff is also stored in the audit log so the proposed change is recoverable even if the user later reverts the file.

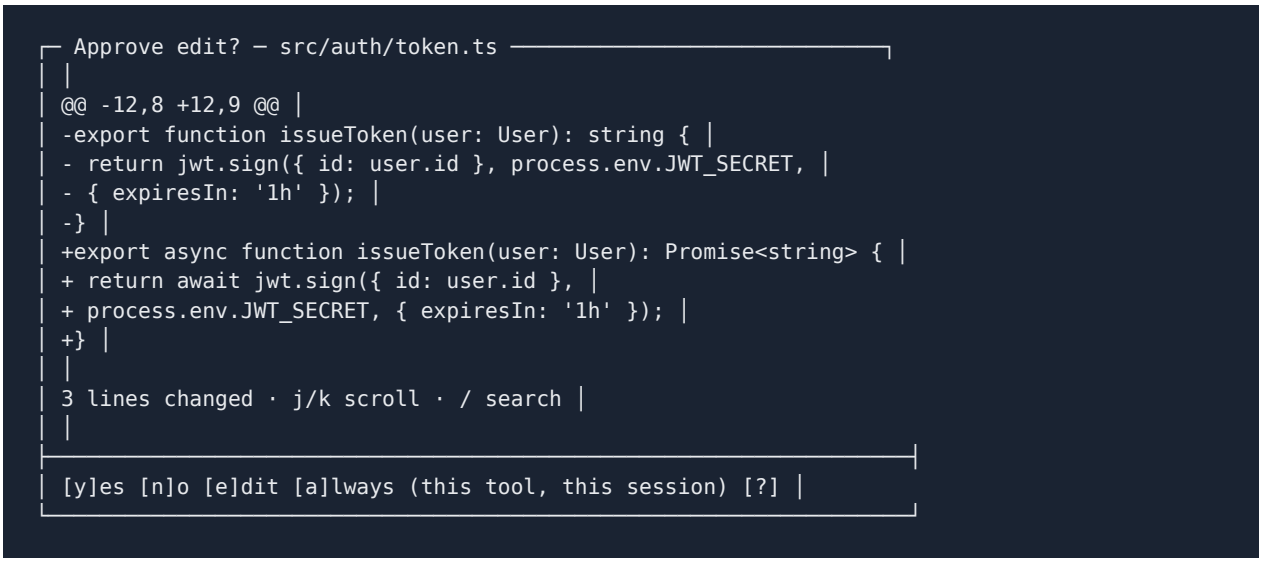

Figure 9.1 — Diff approval prompt for an edit tool call.

9.4 Shell Command Classification

Shell commands receive the most scrutiny because they can do anything. Every command is classified into one of four risk tiers before any policy rule is evaluated. The tier determines the default behavior; the user's policy can tighten (e.g. demote a tier-2 command to tier-3) but never loosen (e.g. promote a tier-4 command to tier-1).

Tier	Risk	Examples	Default Behavior
1	Read-only, in-workspace	ls, cat, head, tail, wc, git status, git log	Auto-approved if in allowlist
2	Read-only, side-effect-free network	curl GET, dig, nslookup, ping	Always prompt (default) or auto if allowlisted
3	Mutating, reversible	git commit, git checkout, npm install, mkdir, touch	Always prompt
4	Mutating, irreversible or destructive	rm -rf, git push --force, git reset --hard, mkfs, dd	Always prompt; some patterns always denied

9.5 Headless Mode (YOLO / Force)

In headless mode (--yolo or --force), the interactive approval prompt is replaced by an auto-approval policy. The two flags differ in scope: --force skips interactive confirmations but still respects the config denylist (it auto-approves everything the user would have been prompted for); --yolo implies --force and additionally bypasses the tool's requiresApproval predicate for tools the user has opted into via config. Neither flag bypasses the hardcoded denylist of destructive shell patterns — those are always enforced.

Flag	Config Allowlist	Config Denylist	Hardcoded Denylist	Tool Predicate
(none)	Auto-approve	Deny	Deny	Respected
--force	Auto-approve	Deny	Deny	Bypassed (treated as allow)
--yolo	Auto-approve	Deny	Deny	Bypassed (treated as allow)

> **Warning.** --yolo is intended exclusively for CI and disposable environments. Using it on a developer workstation with a long-lived repository is an open invitation for the agent to make a change you cannot undo. The audit log still records every action, but the audit log is for forensics, not prevention.

9.6 Audit Log

Every approval decision — whether granted or denied, whether auto or interactive — is written to ~/.sky/audit/audit.log. The log is append-only (no rotation, no compaction); the user is responsible for managing its size via external log rotation if needed. Each entry is a JSON object on a single line, with the following schema.

```
{
  "timestamp": "2026-07-12T09:14:36.100Z",
  "sessionId": "abc123def456",
  "toolCallId": "call_1",
  "toolName": "edit",
  "input": {
    "path": "src/auth/token.ts",
    "oldText": "export function issueToken...",
    "newText": "export async function issueToken..."
  },
  "decision": "prompt",
  "reason": "No auto-approval rule matched",
  "granted": true,
  "autoApproved": false,
  "diff": {
    "path": "src/auth/token.ts",
    "added": 4,
    "removed": 4,
    "sha256": "a3f5..."
  }
}
```

Figure 9.2 — Audit log entry for an edit tool call.

9.7 Approval Workflow Diagram

The end-to-end approval workflow for a single tool call proceeds through the stages shown below. Each stage is a pure function of its inputs and can be unit tested in isolation.

Figure 9.3 — Approval workflow. Every stage writes to the audit log; the tool only executes if all gates pass.

9.8 Session-Allowlist Persistence

When the user chooses “always” (a) at an approval prompt, a derived pattern is added to the session allowlist for the current tool. The pattern is the most specific that would have auto-approved the current call: for read it’s a path glob (e.g. src/**/*.ts); for shell it’s a command prefix (e.g. npm test*); for edit it’s a path glob. Session allowlists are written to the session file and persist across resumes, but they do not propagate to other sessions or to the global config. The user can promote a session allowlist to the global config via /save-policy.

10. Testing Strategy

Sky's test strategy is layered to catch defects at the cheapest possible level. Unit tests verify individual modules in isolation; integration tests verify module boundaries; end-to-end tests verify the full agent loop against a fixture workspace. The target coverage is 85% line coverage across the src/ tree, with 100% coverage of the safety and config modules (which have the highest cost of failure). Tests run on every push and every pull request via GitHub Actions; no PR can be merged with failing tests or coverage below the threshold.

10.1 Test Pyramid

Layer	Count (target)	Runtime (target)	What It Verifies
Unit	~1,200	< 30s	Individual functions and classes; pure logic; no I/O
Integration	~250	< 3 min	Module boundaries; agent loop with mocked provider; tool execution
End-to-end	~40	< 15 min	Full agent loop against real provider (recorded fixtures) and fixture workspace
Smoke	~10	< 5 min	Release verification: install, init, basic agent run on a clean machine

10.2 Unit Testing

Unit tests use Vitest, which is chosen for its native ESM support, fast startup, and Jest-compatible API. Each module has a co-located *.test.ts file. Tests are pure: they do not touch the filesystem, network, or real subprocesses. Where a module depends on I/O, the dependency is mocked via vi.mock() or injected via a constructor parameter.

10.3 Integration Testing

Integration tests verify that modules compose correctly. They use real implementations for everything except the LLM provider (which is mocked) and the filesystem (which is sandboxed under a temp directory). The integration test harness spins up an in-memory session store, a real tool registry, a real policy engine, and a mock provider that replays canned responses. This catches defects in wiring, message format translation, and tool dispatch that unit tests miss.

10.4 End-to-End Testing

End-to-end tests run the full Sky binary against a fixture workspace and a recorded LLM response. The LLM is not called live; instead, the test uses a fixture file captured from a previous real run and replays it via the MockProvider. This makes E2E tests deterministic, free, and fast, while still exercising the full stack from CLI entry point through agent loop to tool execution. Fixtures are committed under test/e2e/fixtures/ and refreshed quarterly.

10.5 Test Fixtures

Fixtures are JSON files that capture a real LLM's response to a specific prompt. They are recorded using the sky fixture record command (developer-only), which runs a real agent session and writes every provider chunk to a file. Fixtures include the model name, the prompt, and the streamed response; they do not include API keys or session metadata. Fixtures are version-controlled and reviewed in PRs; a fixture change is a signal that the agent's behavior has shifted and may need attention.

10.6 Coverage Targets

Module	Line Coverage Target	Branch Coverage Target	Notes
src/safety/	100%	100%	Highest cost of failure
src/config/	100%	95%	Validation logic is critical
src/tools/	90%	85%	Per-tool; shell tool gets extra cases
src/agent/	90%	85%	Loop logic; event ordering
src/session/	90%	85%	Migration paths need full coverage
src/llm/	85%	80%	Adapters; retry logic
src/tui/	70%	60%	Hard to test; covered by E2E
src/cli/	70%	60%	Command wiring; covered by E2E
Overall	85%	80%	Enforced in CI

10.7 CI/CD Pipeline

The CI pipeline runs on every push and every PR. It consists of five jobs, each of which must pass for the PR to be mergeable. Jobs run in parallel where possible; the slowest job (E2E) gates the merge.

Job	Triggers	Steps	Timeout
lint	push, PR	eslint, prettier --check, tsc --noEmit	5 min

Job	Triggers	Steps	Timeout
unit	push, PR	vitest run --coverage (unit only)	10 min
integration	push, PR	vitest run (integration suite)	15 min
e2e	PR (not push)	vitest run (e2e suite) on Linux + macOS	30 min
security	nightly, PR	npm audit, license check, SAST scan	10 min
release	tag push	Build, sign, publish to npm, create GitHub release	20 min

10.8 Performance Testing

A weekly performance regression job measures cold-start time, agent loop latency for a standard set of prompts, and memory usage over a long session. Results are tracked over time and posted to a dashboard. A 10% regression on any metric triggers an automatic issue; a 25% regression blocks the next release until investigated.

Metric	Baseline	Threshold (warn)	Threshold (fail)
Cold start (sky --version)	420 ms	550 ms	800 ms
First token latency (gpt-4o)	1.2 s	1.6 s	2.5 s
Tool call overhead (read)	12 ms	20 ms	40 ms
Memory after 100 turns	85 MB	120 MB	200 MB
Session save time (1000 msgs)	45 ms	80 ms	150 ms

11. Error Handling and Logging

Production-grade error handling and observability are what separate a hobby project from a tool engineers can trust. Sky treats every error as a typed, recoverable event with a stable code, a clear user message, and a structured log entry. This section specifies the error taxonomy, the logging architecture, the recovery patterns, and the observability hooks that allow operators to monitor Sky in production environments.

11.1 Error Taxonomy

Every error in Sky is an instance of SkyError, which extends Error with a stable code, an optional cause, and a retryable flag. The codes are partitioned by module prefix; the full catalog is in Appendix B. The typed hierarchy lets the agent loop, the TUI, and the CLI exit-code handler all switch on the same discriminant.

11.2 Recovery Patterns

Different error classes call for different recovery strategies. The agent loop implements a small decision tree that maps error codes to behaviors. The goal is to keep the agent useful in the face of transient failures while failing fast on errors that indicate a fundamental problem (bad config, corrupt session, unreachable workspace).

Error Class	Representative Codes	Recovery	User Experience
Provider (retriable)	SKY-E-5001 (429), SKY-E-5002 (503)	Exponential backoff, 4 attempts; fallback provider if configured	Brief "retrying..." indicator; continues automatically
Provider (non-retriable)	SKY-E-5010 (400), SKY-E-5011 (401)	Stop turn, surface error to user	Red error banner with actionable message
Tool (recoverable)	SKY-E-3020 (oldText not found)	Re-read file, retry once with refreshed content	Brief "re-reading..." indicator; transparent
Tool (unrecoverable)	SKY-E-3040 (denylist), SKY-E-3999 (unexpected)	Stop turn, report to user, ask how to proceed	Error banner with options: retry / skip / abort
Session (corrupt)	SKY-E-4001 (no migration), SKY-E-4002 (parse fail)	Offer to restore from .bak; refuse to start if no backup	Modal dialog with restore option
Config (invalid)	SKY-E-1001 (parse fail), SKY-E-1002 (missing key)	Refuse to start; offer to open $EDITOR	Pre-flight error with file:line pointer
Context (over budget)	SKY-E-2001 (exceeded window)	Offer to compact session or start new one	Modal with compact/new/abort options

11.3 Logging Architecture

Logging uses pino, a high-performance structured JSON logger. The logger is created once at startup and injected into every module via a Logger interface. Logs go to two sinks: a rotating file (~/.sky/logs/sky.log, rotated daily, retained 30 days) and stderr (when --verbose is set). A redactor strips known secret patterns before any log line is written, so API keys and bearer tokens never reach disk.

11.4 Log Levels and Usage

Level	When to Use	Example	Default Output
trace	Per-iteration loop state (dev only)	loop.iteration { n: 3, tokens: 1240 }	Off (file only when SKY_LOG_LEVEL=trace)
debug	Detailed diagnostic info	tool.execute { name: "read", path: "..." }	File only; stderr with --verbose
info	Normal operation milestones	session.started { id: "abc", mode: "agent" }	File; stderr with --verbose
warn	Unexpected but recoverable	provider.retry { attempt: 2, status: 429 }	File + stderr
error	Operation failed, user impacted	tool.failed { name: "shell", code: "SKY-E-3040" }	File + stderr + TUI banner
fatal	Cannot continue, process will exit	config.invalid { file: "~/.sky/config.json" }	File + stderr + exit

11.5 Structured Log Schema

Every log line is a JSON object with a fixed set of top-level fields plus a module-specific msg or data payload. The fixed fields allow log aggregators to index consistently; the payload carries the details.

```
{
  "level": "info",
  "time": "2026-07-12T09:14:32.001Z",
  "pid": 12345,
  "version": "1.0.0",
  "sessionId": "abc123def456",
  "module": "agent.loop",
  "event": "turn.start",
  "msg": "Starting agent turn",
  "data": {
    "mode": "agent",
    "model": "claude-3-5-sonnet",
    "provider": "anthropic",
    "cwd": "/home/alice/projects/api",
    "messages": 12,
    "tokensUsed": 4231,
    "tokensRemaining": 195769
  }
}
```

Figure 11.1 — Structured log entry. Every line is self-contained and parseable by any JSON-aware tool.

11.6 Observability Hooks

For users running Sky in fleet environments (e.g. a CI farm with hundreds of parallel agents), Sky exposes observability hooks that integrate with standard monitoring stacks. The hooks are opt-in and add no overhead when not configured.

Hook	Configuration	Output	Use Case
OpenTelemetry traces	observability.otlpEndpoint	OTLP HTTP spans to a collector	Distributed tracing across provider + tools
Metrics (Prometheus)	observability.metricsPort	Prometheus scrape endpoint	Counter of tool calls, errors, tokens
Webhook on completion	observability.webhook.url	POST JSON payload on session end	Slack/Discord notification on long runs
Sentry (errors)	observability.sentryDsn	Error events to Sentry	Crash reporting for fleet deployments

11.7 Crash Recovery

Sky is designed to survive crashes without losing session state. Every state change is written to the session file atomically before the agent proceeds to the next step. If Sky is killed (SIGKILL, OOM, power loss) mid-turn, the session file on disk reflects the last completed step; the in-flight step is lost, but the session is consistent and resumable. The next sky resume will load the session, detect that the last turn was interrupted (via a lastTurnInterrupted flag set at turn start and cleared at turn end), and offer to roll back any tool calls that completed but whose results were not yet acknowledged by the assistant.

> **Implementation detail.** The atomic write uses the standard temp-file + rename pattern: write to ~/.sky/sessions/<id>.json.tmp, fsync, then rename to <id>.json. On POSIX systems, rename is atomic with respect to the directory entry, so a reader either sees the old file or the new file, never a partial write. On Windows, the same pattern works but requires an additional MoveFileEx with MOVEFILE_REPLACE_EXISTING.

11.8 User-Facing Error Messages

Every error that reaches the user is rewritten into a plain-English message with three parts: what happened, why, and what to do next. The raw error code is shown in brackets for bug reports. Technical details (stack trace, raw provider response) are hidden by default and revealed with --verbose or by pressing d on an error banner in the TUI.

```
# Good error message
[SKY-E-5011] Authentication failed with the OpenAI API.
The API key in OPENAI_API_KEY was rejected (401).
To fix this, verify the key at https://platform.openai.com/api-keys
and update it with: `sky config set providers.openai.apiKeyEnv OPENAI_API_KEY`
(d) for details, (r) to retry, (q) to quit.

# Bad error message (rejected in code review)
Error: Request failed with status 401
    at OpenAIProvider.<anonymous> (.../openai.ts:45:15)
    at processTicksAndRejections (node:internal/process/task_queues:96:5)
```

12. Documentation Requirements

Documentation is a first-class deliverable for Sky, not an afterthought. Every public surface — command, flag, config key, tool, API endpoint — has corresponding documentation that is built, tested, and shipped with the binary. This section specifies the documentation set, the authoring workflow, and the quality bar for each artifact.

12.1 Documentation Set

Artifact	Audience	Location	Format	Update Cadence
README.md	New users	repo root	Markdown	Every release
User guide	Regular users	docs/guide/	Markdown + mdsvex	Every release
CLI reference	All users	docs/cli/	Markdown (auto-generated from Commander)	Every release
Config reference	Power users	docs/config.md	Markdown (auto-generated from Zod schema)	Every release
API reference (TS)	Contributors	docs/api/	TypeDoc HTML	Every PR that touches src/
HTTP API reference	Integrators	docs/http-api.md	Markdown + OpenAPI spec	When --serve changes
Architecture decision records	Maintainers	docs/adr/	Markdown (MADR template)	On architecture change
Contributor guide	Contributors	docs/contributing.md	Markdown	Quarterly review
Changelog	All users	CHANGELOG.md	Markdown (Keep a Changelog format)	Every release
Security policy	All users	SECURITY.md	Markdown	Annual review
Man pages	Unix users	man/sky.1	roff (generated from CLI ref)	Every release

12.2 README Requirements

The README is the first thing a new user sees. It must answer four questions in the first screen of content: **What is this? Why should I use it? How do I install it? How do I use it?** Everything else — architecture, contributing, license — goes below the fold or in linked docs.

The README must include: a one-paragraph elevator pitch; a 30-second demo GIF or asciinema; install instructions for npm, Homebrew, and binary download; a minimal usage example; a pointer to the full docs; the project's license and contribution policy. The README is rendered to HTML on the GitHub repo home

page and on the project website; both renderings must look correct.

12.3 User Guide Structure

12.4 API Documentation

The TypeScript API is documented via TSDoc comments in the source and rendered via TypeDoc to docs/api/. Every exported function, class, interface, and type has a TSDoc comment with a description, parameter descriptions, and at least one example. The TypeDoc build runs in CI; missing TSDoc on a newly exported symbol fails the build. The rendered HTML is published to the project website on every release.

12.5 Architecture Decision Records

Every non-trivial architectural decision is recorded as an ADR in docs/adr/ using the MADR template. An ADR is required for: adding or removing a runtime dependency, changing a public interface, altering the safety model, or introducing a new module. ADRs are immutable once merged; supersession happens via a new ADR that references the old one.

12.6 Changelog Policy

The changelog follows the Keep a Changelog format. Every PR that changes user-facing behavior must include a changeset describing the change. Changesets are managed via @changesets/cli; on release, the changesets are consumed and the CHANGELOG.md is regenerated. The format groups changes into Added, Changed, Deprecated, Removed, Fixed, and Security sections, with each entry linking to the PR that introduced it.

12.7 Documentation Testing

Code samples in documentation are tested. Every fenced code block in docs/ is extracted by a test runner and executed against a fixture workspace; failures fail the CI build. This prevents the most common documentation defect: examples that no longer work because the codebase moved under them. The test harness respects a <!-- no-test --> comment for examples that are intentionally illustrative rather than runnable.

13. Development Roadmap

Sky is developed in five phases, each with explicit scope, exit criteria, and a launch checklist. The phases are sequential: a later phase cannot start until the prior phase's exit criteria are met. This discipline is intentional — it prevents the "80% done forever" failure mode where every release adds features but none is solid enough to call v1.

13.1 Phase Overview

Phase	Duration	Theme	Deliverable
Phase 1	4 weeks	MVP: core CLI + single provider	sky agent + sky ask + read/write/edit tools, OpenAI only
Phase 2	4 weeks	Multi-provider + sessions	Anthropic + Ollama adapters, sky resume + sky ls
Phase 3	3 weeks	Safety + MCP	Approval workflow, audit log, MCP proxy, sky mcp
Phase 4	3 weeks	Plan mode + headless + polish	sky plan, --yolo/--force, sky --serve, performance pass
Phase 5	2 weeks	Docs + v1.0 release	User guide, API docs, ADRs, launch checklist
Post-v1	ongoing	Extensions	More providers, more tools, community MCP servers

13.2 Phase 1: MVP (Weeks 1-4)

Goal: prove the core thesis. A user can install Sky, configure it with an OpenAI API key, start an agent session in a directory, and have the agent read, write, and edit files with explicit approval. Everything else (multi-provider, sessions, MCP, plan mode, headless) is out of scope for Phase 1 and explicitly deferred.

Week	Focus	Exit Criterion
1	Project scaffold, CLI entry, config loader, Zod schema	sky --version and sky init work
2	Agent loop, OpenAI adapter, read tool	Agent can read a file and report its contents
3	Write + edit tools, approval UI (diff viewer)	Agent can edit a file end-to-end with approval
4	TUI polish, error handling, basic tests	Smoke test passes; npm test green

13.3 Phase 2: Multi-Provider and Sessions (Weeks 5-8)

Goal: remove the single-provider lock-in and make sessions first-class. After Phase 2, a user can switch between OpenAI, Anthropic, and Ollama by changing config, and can resume a previous session. The provider abstraction introduced here is the foundation for fallback (Phase 4) and OpenRouter (Phase 4).

Week	Focus	Exit Criterion
5	Provider interface, refactor OpenAI to adapter	OpenAI adapter behind interface; tests pass
6	Anthropic adapter, Ollama adapter	All three providers functional
7	Session store, sky resume, sky ls	Can resume a session across process restarts
8	Context window management, token counting	Long sessions auto-trim without errors

13.4 Phase 3: Safety and MCP (Weeks 9-11)

Goal: harden the safety layer and open the tool ecosystem via MCP. After Phase 3, every destructive action is gated by a typed policy, every decision is audited, and external tools can be plugged in without forking Sky.

Week	Focus	Exit Criterion
9	Policy engine, denylist enforcement, audit log	All tool calls logged; denylist blocks destructive ops
10	MCP client, sky mcp commands, mcp_proxy tool	Can register and call an external MCP server
11	Shell + git tools with tiered classification	Shell tool classifies commands correctly in 100% of test cases

13.5 Phase 4: Plan Mode, Headless, and Polish (Weeks 12-14)

Goal: round out the feature set for v1 and prepare for production use. Plan mode adds the design-first workflow; headless mode unlocks CI use cases; the HTTP server enables editor integrations. The performance pass ensures cold-start and per-turn latency meet the success criteria from Section 1.

Week	Focus	Exit Criterion
12	Plan mode (clarification, planning, deferred execution)	sky plan completes a full design-then-execute flow
13	Headless mode (--yolo, --force), --json output, --serve	CI run produces deterministic file changes; HTTP API serves events
14	Performance pass, memory leak hunt, error message polish	All performance thresholds from Section 10.8 met

13.6 Phase 5: Documentation and v1.0 Release (Weeks 15-16)

Goal: ship a v1.0 that a senior engineer can adopt without reading the source. Every public surface is documented, every example is tested, every ADR is written. The launch checklist is a gate; no item can be deferred to v1.1.

Week	Focus	Exit Criterion
15	User guide, CLI reference, config reference, API docs	All docs written and tested; TypeDoc green
16	Release candidate, bug bash, final polish, v1.0.0 tag	Launch checklist 100% complete; v1.0.0 published to npm

13.7 Launch Checklist

The following must all be true before v1.0.0 is tagged. Items are tracked in a launch-checklist.md file in the repo; each item has an owner and a due date.

Category	Item	Owner	Status Gate
Code	All Phase 1-4 exit criteria met	Tech lead	Required
Code	Test coverage ≥ 85% overall, 100% safety/config	QA	Required
Code	No open P0/P1 bugs	Tech lead	Required
Code	Performance thresholds met (Section 10.8)	Perf owner	Required
Docs	README answers 4 questions in first screen	Docs lead	Required
Docs	User guide complete with runnable examples	Docs lead	Required
Docs	API docs green in CI	Docs lead	Required
Docs	All ADRs merged	Tech lead	Required
Release	CHANGELOG.md updated for v1.0.0	Release manager	Required
Release	npm package signed and published	Release manager	Required
Release	GitHub release notes drafted	Release manager	Required
Release	Website updated with v1.0 install	Docs lead	Required
Security	npm audit clean (0 high/critical)	Security owner	Required
Security	License review complete	Legal	Required
Security	Security policy published (SECURITY.md)	Security owner	Required
Community	Contribution guide published	Community lead	Required
Community	Issue templates + PR template live	Community lead	Required
Community	Code of Conduct published	Community lead	Required

13.8 Post-v1 Roadmap (Indicative)

After v1.0, development continues on a quarterly cadence. The themes below are indicative, not committed; each quarter's actual scope is decided in a planning meeting at the start of the quarter.

Quarter	Theme	Representative Work
Q1 post-v1	More providers	Mistral, Google Gemini, Azure OpenAI, AWS Bedrock
Q2 post-v1	More tools + MCP ecosystem	LSP-aware search, structured grepping, community MCP registry
Q3 post-v1	Performance + scale	Streaming diffs, parallel tool calls, context cache
Q4 post-v1	Enterprise features	SSO, audit log streaming, team config, on-prem provider gateway

> **Roadmap philosophy.** The roadmap is a plan, not a contract. The maintainers reserve the right to reprioritize based on user feedback, security disclosures, or ecosystem shifts. What does not change is the commitment to the design principles in Section 1.2 and the non-goals in Section 1.5: any proposal that violates either requires an ADR and maintainer consensus, regardless of which quarter it lands in.

Appendix A: Configuration Reference

This appendix lists every configuration key in ~/.sky/config.json, its type, default value, and valid values. The schema is auto-generated from src/config/schema.ts and exported to ~/.sky/config.schema.json on first run for editor autocomplete.

A.1 Top-Level Keys

Key	Type	Default	Description
schemaVersion	integer (literal 1)	1	Config schema version; migrated automatically
defaultProvider	enum	openai	Provider used when --provider not passed
defaultModel	string	(none; required)	Model used when --model not passed
providers	object	(see A.2)	Per-provider configuration
tools	object	(see A.3)	Per-tool policy
tui	object	(see A.4)	TUI theme and layout
sessions	object	(see A.5)	Session management
logging	object	(see A.6)	Logging configuration
mcp	object	(see A.7)	MCP server registrations
observability	object	(see A.8)	Traces, metrics, webhooks

A.2 providers.*

Key	Type	Default	Description
apiKeyEnv	string	(none)	Name of env var holding the API key (preferred over apiKey)
apiKey	string	(none)	Literal API key (discouraged; logged as warning)
baseUrl	URL string	(provider default)	Override for OpenAI-compatible endpoints
defaultModel	string	(required)	Model to use when none specified
models	object	(none)	Per-model metadata (context window, cost)
fallback	object	(none)	Fallback provider config: {provider, model, triggerAfter}

A.3 tools.*

Key	Type	Default	Description
read.autoApprove	string[] (globs)	[]	Path globs auto-approved for read
read.deny	string[] (globs)	[".env*", "credentials*", "*.pem", "*.key"]	Path globs always denied for read
write.allowOutsideCwd	boolean	false	Allow writes outside session cwd
write.autoApprove	string[] (globs)	[]	Path globs auto-approved for write
edit.autoApprove	string[] (globs)	[]	Path globs auto-approved for edit
shell.autoApprove	string[] (patterns)	[]	Command patterns auto-approved for shell
shell.deny	string[] (patterns)	["rm -rf /", "mkfs.*", "dd of=/dev/*", "shutdown", "reboot"]	Command patterns always denied
shell.env	object	(none)	Env vars injected into every shell call
git.allowForcePush	boolean	false	Allow git push --force (otherwise always denied)
git.autoApproveReads	boolean	true	Auto-approve git status/diff/log/branch

A.4 tui.*

Key	Type	Default	Description
theme.colors.accent	string (color name)	cyan	Color for active elements
theme.colors.success	string	green	Color for success states
theme.colors.error	string	red	Color for errors
theme.colors.warning	string	yellow	Color for warnings
theme.colors.info	string	blue	Color for info
theme.colors.planning	string	magenta	Color for search/plan
theme.glyphs.indicator	string (1 char)	⬢	Status indicator glyph
theme.layout.submitOnEnter	boolean	true	Submit on Enter (last line); Shift+Enter always newline
theme.layout.showTokenBar	boolean	true	Show token usage in status bar
theme.layout.compactMode	boolean	false	Force compact mode (narrow terminals)

A.5 sessions.*

Key	Type	Default	Description
autoCompact	boolean	true	Automatically compact sessions exceeding threshold
autoCompactThreshold	integer (tokens)	50000	Token count above which auto-compaction triggers
retentionDays	integer	90	Days to retain paused sessions before archiving

A.6 logging.*

Key	Type	Default	Description
level	enum (trace/debug/info/warn/error)	info	Minimum log level for file sink
fileRetentionDays	integer	30	Days to retain rotated log files

A.7 mcp.servers[]

Key	Type	Default	Description
name	string	(required)	Unique server name; used in tool prefix
command	string	(required)	Executable to launch the MCP server
args	string[]	[]	Arguments passed to command
env	object	(none)	Env vars for the server process
approvalMode	enum (auto/manual/deny)	manual	Approval policy for tools exposed by this server

A.8 observability.*

Key	Type	Default	Description
otlpEndpoint	URL string	(none)	OTLP HTTP endpoint for OpenTelemetry traces
metricsPort	integer	(none)	Port for Prometheus metrics scrape endpoint
webhook.url	URL string	(none)	Webhook POSTed on session completion
sentryDsn	string	(none)	Sentry DSN for crash reporting

A.9 Environment Variables

Variable	Purpose	Default
SKY_CONFIG	Path to config file	~/.sky/config.json
SKY_LOG_LEVEL	Override log level	from config
SKY_AUTO_APPROVE_SHELL	Auto-approve all shell commands	unset (false)
SKY_PROVIDERS_OPENAI_API_KEY	OpenAI API key (fallback)	unset
SKY_PROVIDERS_ANTHROPIC_API_KEY	Anthropic API key (fallback)	unset
SKY_NO_COLOR	Disable ANSI color	unset
SKY_FORCE_COLOR	Force ANSI color	unset
NO_COLOR	Standard no-color env var	unset
EDITOR	Editor for sky config and edit approval	vi

Appendix B: Error Code Catalog

Every error in Sky carries a stable code in the format SKY-E-XXXX. Codes are partitioned by module prefix. The table below is the complete catalog as of v1.0.0; new codes may be added in minor releases, but existing codes never change meaning. CI scripts can switch on these codes to implement provider-specific or tool-specific recovery logic.

B.1 Config Errors (1xxx)

Code	Message	Retryable	Exit Code
SKY-E-1000	Config file not found. Run `sky init` to create one.	No	64
SKY-E-1001	Config file failed to parse: {detail}	No	64
SKY-E-1002	No API key configured for provider {name}.	No	64
SKY-E-1003	Config schema validation failed: {fields}	No	64
SKY-E-1004	Unknown provider: {name}	No	64
SKY-E-1005	Unknown model: {name} for provider {provider}	No	64
SKY-E-1010	Config key not found: {key}	No	64
SKY-E-1011	Config key has wrong type: {key} expected {expected}	No	64
SKY-E-1020	Config migration failed: {detail}	No	64

B.2 Agent / Context Errors (2xxx)

Code	Message	Retryable	Exit Code
SKY-E-2000	Agent loop aborted by user (Ctrl-C)	No	130
SKY-E-2001	Context window exceeded. Run `/compact` or start a new session.	No	68
SKY-E-2002	No tool definitions provided but agent requested tool call.	No	70
SKY-E-2003	Agent turn exceeded max iterations ({n}).	No	70
SKY-E-2010	Plan mode rejected tool call: {name}.	No	0
SKY-E-2011	Ask mode received tool call (should be filtered).	No	70

B.3 Tool Errors (3xxx)

Code	Message	Retryable	Exit Code
SKY-E-3000	Unknown tool: {name}	No	70
SKY-E-3001	Tool input validation failed: {detail}	Yes	0
SKY-E-3002	Tool output validation failed (internal bug)	No	70
SKY-E-3010	Write refused: path outside cwd ({path})	Yes	0
SKY-E-3020	Edit failed: oldText not found in {path}	Yes	0
SKY-E-3021	Edit failed: oldText ambiguous ({n} matches)	Yes	0
SKY-E-3030	Search failed: ripgrep error: {detail}	No	0
SKY-E-3040	Shell command denied (denylist): {command}	No	0
SKY-E-3041	Shell command timed out after {n}ms	No	0
SKY-E-3050	Git force push denied by policy	No	0
SKY-E-3060	MCP server {name} is in deny mode	No	0
SKY-E-3061	MCP server {name} not connected	No	0
SKY-E-3999	Unexpected tool error: {detail}	No	70

B.4 Session Errors (4xxx)

Code	Message	Retryable	Exit Code
SKY-E-4000	Session not found: {id}	No	65
SKY-E-4001	Session schema migration failed: {detail}	No	65
SKY-E-4002	Session file corrupt: {detail}	No	65
SKY-E-4010	Session is read-only (--view mode)	No	2
SKY-E-4020	Session index corrupt; rebuilding	No	0

B.5 Provider Errors (5xxx)

Code	Message	Retryable	Exit Code
SKY-E-5000	Provider request failed: {detail}	Yes	66
SKY-E-5001	Provider rate limited (429)	Yes	66
SKY-E-5002	Provider temporarily unavailable (503)	Yes	66
SKY-E-5003	Provider timeout after {n}ms	Yes	66
SKY-E-5010	Provider bad request (400): {detail}	No	66
SKY-E-5011	Provider authentication failed (401)	No	66
SKY-E-5012	Provider forbidden (403): {detail}	No	66
SKY-E-5013	Provider requested content filter (451)	No	66
SKY-E-5020	Provider response stream interrupted	Yes	66
SKY-E-5030	Provider stream parse error: {detail}	No	66
SKY-E-5040	Provider cost budget exceeded ({spent} > {budget})	No	66
SKY-E-5099	Unknown provider error: {detail}	No	66

B.6 Safety Errors (6xxx)

Code	Message	Retryable	Exit Code
SKY-E-6000	User denied approval for tool call: {name}	No	67
SKY-E-6001	Approval timed out after {n}s	No	67
SKY-E-6010	Policy violation: {detail}	No	67
SKY-E-6020	Audit log write failed: {detail}	No	70

B.7 TUI Errors (7xxx)

Code	Message	Retryable	Exit Code
SKY-E-7000	Terminal too narrow (min 60 cols)	No	70
SKY-E-7001	Terminal does not support color (use --no-color)	No	70
SKY-E-7010	TUI render error: {detail}	No	70

B.8 CLI Errors (8xxx)

Code	Message	Retryable	Exit Code
SKY-E-8000	Unknown command: {name}	No	2
SKY-E-8001	Missing required argument: {name}	No	2
SKY-E-8002	Invalid flag value: {flag}={value}	No	2
SKY-E-8010	Cannot start Sky: another instance holds the lock	No	1
SKY-E-8099	Internal error (please file a bug): {detail}	No	70

Stability guarantee. Error codes follow semantic versioning. Codes never change meaning in a minor or patch release; they may be *added* in minor releases and *deprecated* (but not removed) in major releases. CI scripts that switch on codes are safe across minor and patch updates.